

Communication Lower Bounds for Multi-Party Graph Traversal

Quarter 2 Project Report

Ryan Batubara
rbatubara@ucsd.edu

Ivy Hawks
mahawks@ucsd.edu

Darren Ho
dah103@ucsd.edu

Ciro Zhang
ciz001@ucsd.edu

Mentor: Dr. Shachar Lovett
slovett@ucsd.edu

Abstract

A large number of computational problems can be modelled as communication games: For example, given a graph, two players each have a subgraph they know is traversable, and wish to collaboratively construct a path from one vertex to another. Communication complexity studies such situations by creating asymptotic bounds on the minimum communication necessary to complete these tasks. In this report, we prove communication lower and upper bounds for a variety of graph traversal problems, such as the example above. We also review potential applications of these bounds, such as for modelling real-world scenarios and designing algorithms.

Website: <https://rybplayer.github.io/capstone/>
Repository: <https://github.com/rybplayer/DSCCapstone>

1	Introduction	2
2	The Two-Party Traversal Problem	5
3	The Iterated Two-Party Traversal Problem	9
4	The Multi-Party Traversal Problem	9
5	The Iterated Multi-Party Traversal Problem	10
6	Other Problems	10
	References	10
	Appendices	A1
A	Multi-Party Graph Traversal Case Studies	A1
B	Weekly Contributions	A2
C	Updated Proposal	A4

1 Introduction

1.1 A motivating problem

Graphs are fundamental data structures for modelling relationships between entities. For example, an airline may choose to store their flights as a graph, whose vertices are destinations and directed edges are flights between destinations. Many well-known algorithms make this a feasible data structure for the airline. Now suppose a user wishes to plan flights, perhaps with transits, from point A to B . This user may need to consider a large variety of airlines, each with their own graph. How can the user do this?

Clearly, the user can get the full database of all flights, and compute the desired sequence of flights on their own. But this is obviously costly in many regards. We want to minimize the amount of processing and communication between these parties while still giving the user the desired result. Indeed, there exists a variety of algorithms for this in the literature.

In this report, we ask an adjacent question: What is the minimum amount of communication necessary for the user to complete this task, without considering the local storage and computational cost to each party? Yao's communication complexity model formalizes this idea, allowing us to prove algorithmic lower bounds for situations like the above. These algorithmic lower bounds are concrete, mathematical claims that show we cannot asymptotically reduce communication beyond a certain point.

1.2 Why communication is the bottleneck

In the model we consider, each party has unlimited local computation and we only wish to minimize the communication between them. In the airline routing example above, this restriction is quite realistic: airline servers and user computers can easily compute routes given the data. However, neither can afford to broadcast or receive all states due to bandwidth limitations, especially if there are multiple requests. The communication cost, not the local computation, is the bottleneck.

Communication complexity makes this separation explicit. In particular, it shows that even when each party can compute arbitrarily powerful local summaries, some global questions still require a large number of bits to be exchanged. For example, a route may require flights from multiple airlines. Thus communication is not just a practical nuisance, but an inherent barrier that be designed around.

1.3 Why communication bounds matter

These lower bounds have a variety of applications for algorithm, system and software design. An algorithm's complexity could be described as an upper bound on the amount of communication between parties. If communication complexity provides a corresponding lower bound, this means this algorithm is tight and its communication cannot asymptoti-

cally be improved. For system designers, there is often a lot of choice in how communication is routed through a network. For example, a central server could manage all communications, or a distributed network may be used. Communication complexity can provide lower bounds for each of these scenarios, for instance by proving that distributed networks are asymptotically no better than a centralized communication server. Communication complexity can also inform software decisions, by providing insight into how to manage the bits of communication between parties.

1.4 Communication complexity basics

We briefly define the various measures of communication complexity used in this report. Interested readers should see [Rao and Yehudayoff \(2020\)](#) or [Roughgarden \(2015\)](#) for a more thorough introduction. Let $f : X \times Y \rightarrow \{0, 1\}$ be a function. In the two-party model, Alice receives $x \in X$ and Bob receives $y \in Y$, and they wish to compute $f(x, y)$. A *deterministic protocol* is a binary decision tree whose nodes are labeled by which party speaks and whose edges are labeled by bits 0 or 1. The *transcript* $\pi(x, y)$ is the sequence of bits exchanged for a particular input x and y . The protocol must output $f(x, y)$ at a leaf. The *deterministic communication complexity* $D(f)$ is the minimum, over all correct protocols, of the maximum transcript length (or equivalently, protocol tree depth) over all inputs.

A *nondeterministic protocol* for f has two parts: First, an external party has access to x and y , and provides advice a to both players. Then, the players, with access to a and their respective input, communicate to decide whether to output 0 or 1. Let $\Pi(x, y, a)$ denote the output of the protocol, which we say is correct if

- If $f(x, y) = 1$, then $\exists a$ with $\Pi(x, y, a) = 1$.
- If $f(x, y) = 0$, then $\forall a$, $\Pi(x, y, a) = 0$.

The cost of Π is the sum of a and any communication between players. The *nondeterministic cost* of f , denoted $N^1(f)$, is the minimal cost of a protocol computing f . The *co-nondeterministic cost* of f is the nondeterministic cost for $1 - f$.

Randomized protocols allow parties to use random coins to reduce communication. A public-coin protocol assumes a shared random string, while a private-coin protocol lets each party sample independently. Fixing the randomness yields a deterministic protocol, so a randomized protocol is a distribution over deterministic protocols. The *randomized communication complexity* $R_\epsilon(f)$ is the minimum communicated bits needed to compute f with error at most ϵ on every input. Public-coin ($R_\epsilon^{pub}(f)$) and private-coin ($R_\epsilon(f)$) models are equivalent up to small overhead, and randomness often trades a small probability of error for large savings in communication. Let $R(f) = R_c(f)$ with $c = 1/3 < 1/2$.

1.5 Communication complexity basic examples

Two classic communication complexity problems are equality and disjointness. Equality asks whether $x = y$ for $x, y \in \{0, 1\}^n$, and disjointness asks whether $X \cap Y = \emptyset$ for subsets

of $[n]$. Both are useful for a range of lower-bound techniques, so we go through their communication complexity bounds below:

Claim 1. *Equality, denoted $\text{EQ} : X \times Y \rightarrow \{0, 1\}$, outputs 1 if $x \in X$ and $y \in Y$ have $x = y$, and 0 otherwise. EQ has $D(\text{EQ}) = n$, $N^1(\text{EQ}) = n$, $N^0(\text{EQ}) = \Theta(\log n)$, and $R_\epsilon^{\text{pub}}(\text{EQ}) = O(\log(1/\epsilon))$.*

Proof. First $D(\text{EQ}) = n$.

- Upper bound: Alice sends all n bits of x to Bob, who compared it to y and outputs.
- Lower bound: Any deterministic protocol partitions the $\{0, 1\}^n \times \{0, 1\}^n$ matrix M_{EQ} into combinatorial z -rectangles, i.e. regions $R = A \times B \subseteq X \times Y$ where $f(a, b) = z$ for all $a \in A$, $b \in B$, and $z \in \{0, 1\}$. Clearly $M_{\text{EQ}} = I_{\{0, 1\}^n \times \{0, 1\}^n}$. Hence there are 2^n 1-rectangles or at least 2^n leaves in the protocol tree. Thus $D(f) \geq \log_2(2^n) = n$.

Now $N^1(\text{EQ}) = n$. This means we want to certify $x = y$.

- Upper bound: The prover sends x to both players. Both players check if it matches their input. If so, accept.
- Lower bound: Similar to $D(\text{EQ})$, we need 2^n distinct certificates to cover all possible inputs. Hence certificate length $\geq \log_2(2^n) = n$.

Now $N^0(\text{EQ}) = \log(n)$. This means we want to certify $x \neq y$.

- The prover computes an index $i \in [n]$ where $x_i \neq y_i$. This takes $\log(n)$ bits to communicate to Alice and Bob, who can exchange bits at that coordinate and witness if they differ.
- There are n possible indices i . If certificates had length $< \log(n)$, there would be $< n$ certificates which cannot cover all the possibilities.

Now $R_\epsilon^{\text{pub}}(\text{EQ}) = O(\log(1/\epsilon))$.

- Upper bound: Using shared randomness, pick k strings $r^{(1)}, \dots, r^{(k)} \in \{0, 1\}^n$. For each $j \in [k]$, Alice sends $b_j = \langle x, r^{(j)} \rangle \bmod 2$. Bob computes $\langle y, r^{(j)} \rangle \bmod 2$ and checks equality for all $j \in [k]$. If $x = y$, this always succeeds. If $x \neq y$, each check fails independently with probability $1/2$, since the $r^{(j)}$ are uniformly random. Take $k = \lceil \log_2(1/\epsilon) \rceil$.

We do not prove it, but $R_\epsilon^{\text{pub}}(\text{EQ}) = O(\log(n))$ with only private randomness. $\square$

Claim 2. *Set disjointness, denoted DISJ , interprets $x, y \in \{0, 1\}^n$ as subsets $X, Y \subseteq [n]$ with*

$$\text{DISJ}(x, y) = 1 \iff \forall i \in [n], x_i y_i = 0. \quad (1)$$

Then $D(\text{DISJ}) = \Theta(n)$, $N^1(\text{DISJ}) = \Theta(n)$, $N^0(\text{DISJ}) = \Theta(\log n)$, $R(\text{DISJ}) = \Theta(n)$. Furthermore, if $|X| = |Y| = k \ll n$, then $D(\text{DISJ}) = \Theta(k)$.

These bounds are well known in the literature, so we choose to omit the proof for DISJ bounds. Both EQ and DISJ are very useful for *reductions*, where we embed a problem with known bounds in another, to show this other problem cannot be solved too efficiently. In particular, EQ is “maximally complicated” for deterministic protocols and DISJ is “maximally complicated” for randomized protocols, in the sense that $D(\text{EQ}) = n$ and $R(\text{DISJ}) = \Theta(n)$.

1.6 Literature Review

In this section, we briefly review some results related to our graph traversal problems.

This section is under construction, as we are still considering new variants of our problem. Therefore, we are not yet sure what results in the literature will be relevant to discuss here.

2 The Two-Party Traversal Problem

2.1 Motivating our problem

In this and the following sections, we will define and show communication bounds for a series of problems pertaining to multi-party graph traversal. Please see Appendix A for a variety of case studies that motivates the following problem definitions.

2.2 Problem Statement

We begin with the simplest non-trivial formulation of our problem, inspired by 1.1.

Consider a graph $G = (V, E)$ and subgraphs $G_1, \dots, G_k \subset G$. Let $G_i = (V_i, E_i)$ for these subgraphs. For now let us suppose G is unweighted and undirected. The subgraphs need not cover G and can overlap. Each player i knows G and G_i , but not the other player's graphs. The goal is, given some starting vertex $v \in V$, and some end vertex $w \in V$ to:

1. Identify if it is possible to construct a path $v, v_1, \dots, v_l, w$ such that every edge on the path is in $\bigcup_{i=0}^k E_i$.
2. If such a path exists, find the minimum number of edges needed to get from v to w , while using only edges belonging to $\bigcup_{i=0}^k E_i$.

If some $e = (u, v) \in G_i$, we must have $u, v \in V(G_i)$. But the converse is not always true.

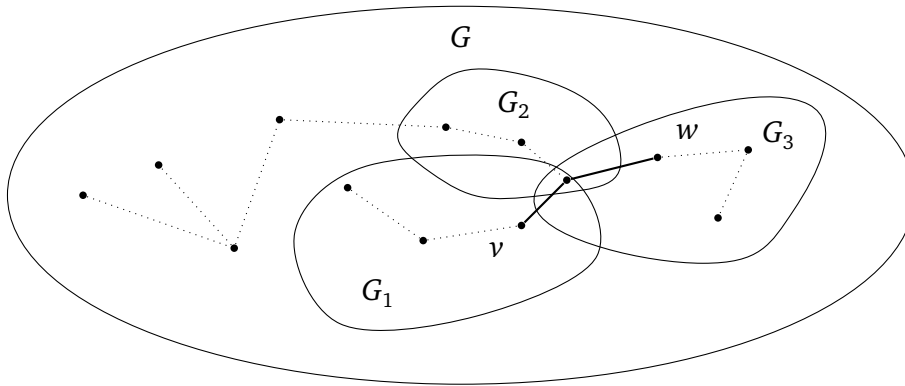


Figure 1: A valid path from v to w in bold. Note that G may contain nodes not belonging to any player. Also, it may be that edge $(u, v) \notin E(G_i)$ though $u, v \in V(G_i)$.

Sometimes, condition 1 fails, as in Figure 2. In such a case, it is sufficient for one party to be convinced that no path exists.

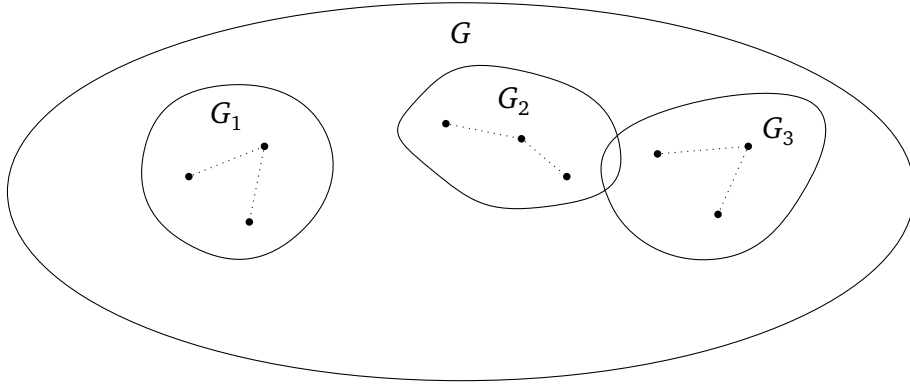


Figure 2: Worst case for case item 1: Disjoint subgraphs G_i

2.3 Two-party lower bounds

We begin by considering the case with only two players and thus two subgraphs. Let these players be named Alice and Bob, with subgraphs G_A and G_B respectively. Denote this problem by TRAV_2 , i.e. the traversal problem for 2 players, where $|V(G)| = n$ and $|E(G)| = O(n^2)$.

Claim 3. Recall that equality EQ has inputs $x, y \in \{0, 1\}^n$, and $\text{EQ}(x, y) = 1$ iff $x = y$. Then $D(\text{TRAV}_2) \geq D(\text{EQ})$ and so on.

Proof. **Equality gadget.** We construct a gadget that enforces equality at a single bit position. The gadget is shown in Figure 2.3. The gadget consists of two parallel branches.

1. The top branch corresponds to the assignment $x_i = y_i = 1$,
2. The bottom branch corresponds to the assignment $x_i = y_i = 0$.

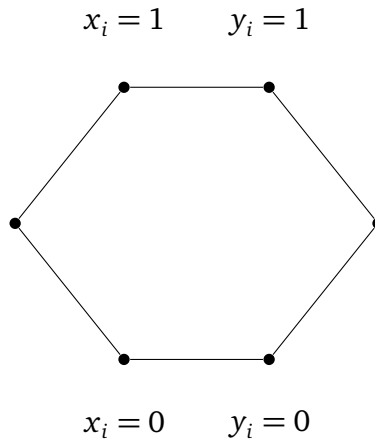


Figure 3: Gadget for proving equality. A path only exists if $x_i = y_i$ for either branch.

The idea is that Alice enables exactly one entry node of the gadget depending on x_i , and Bob enables exactly one exit node depending on y_i . A path exists through the gadget if and only if Alice and Bob select nodes on the same branch, which holds exactly when $x_i = y_i$.

Reduction to our graph problem. Construct a graph of n equality gadgets in a chain, as in Figure 4. Construct subgraph G_A as follows: Include all vertices at the start and end of each equality gadget. Also, for every $i \in [n]$, give the vertex labelled $x_i = 1$ if this is true, else $x_i = 0$. Then, add all edges between these vertices, as well as all edges between $x_i = z$ and $y_i = z$ for $z \in \{0, 1\}$. Do similar for G_B .

In the resulting graph, there exists a path from v to w if and only if all gadgets are traversable, which occurs if and only if $x_i = y_i$ for all i , i.e., if and only if $x = y$.

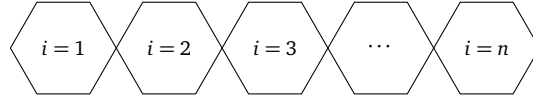


Figure 4: n equality gadgets chained together.

We conclude by noting that the resulting graph G has $\Theta(n)$ vertices and $\Theta(n)$ edges. $\square$

Claim 4. Recall that in two-party DISJ, Alice and Bob are given sets $X, Y \subseteq [n]$ each. The goal is to determine whether $X \cap Y = \emptyset$. We claim $D(\text{TRAV}_2) \geq D(\text{DISJ})$ and so on.

Proof. **Disjointness gadget.** We construct a gadget that enforces disjointness at a single entry $i \in [n]$. The gadget is shown in Figure 5. There, let x_i and y_i be indicator functions whether $i \in X$ and $i \in Y$ respectively. Then:

1. The top branch corresponds to when x_i has the element but y_i does not
2. The middle branch corresponds to when x_i and y_i do not have the element
3. The bottom branch corresponds to when y_i has the element but x_i does not

In other words, there is no path left to right if $i \in X \cap Y$.

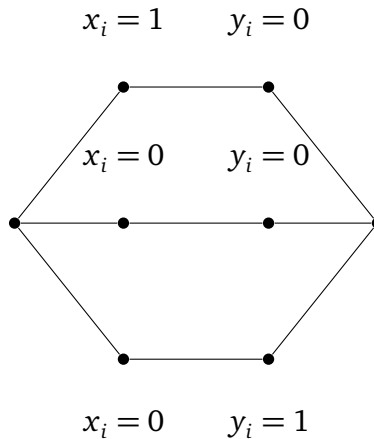


Figure 5: Disjointness gadget. A path left to right only exists if either $x_i = 0$ or $y_i = 0$.

Reduction to our graph problem. We construct a graph G by chaining one copy of this gadget for each index $i \in [n]$ between a global start vertex v (left) and end vertex w (right). Construct G_A by giving all vertices at the start and end of every disjointness gadget, and vertices labelled $x_i = 1$ if it is true, else $x_i = 0$. Add all edges between these vertices, and all edges between $x_i = z$ and $y_i = z'$ for $z, z' \in \{0, 1\}$. Do similar for G_B .

In the resulting graph, there exists a path from v to w if and only if all disjointness gadgets are traversable, which occurs if and only if $X \cap Y = \emptyset$.

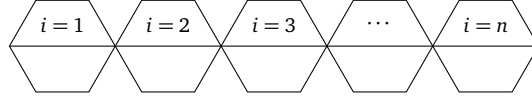


Figure 6: Equality gadgets chained as adjacent hexagonal modules.

We conclude by noting that $|V(G)| = \Theta(n)$ and $|E(G)| = \Theta(n)$. □

2.4 Specialized two-party lower bounds

The previous proofs show that TRAV_2 is maximally complicated in terms of deterministic communication complexity, and almost maximally complicated in terms of randomized communication complexity. A natural follow up question is to ask if we can do better. That is, given some additional constraints, can we solve the problem with less communication asymptotically?

Definition 1. Define the CTRAV_2 problem as a TRAV_2 problem where G_A and G_B are guaranteed to be complete and connected.

Claim 5. $D(\text{CTRAV}_2) \geq D(\text{DISJ})$ and so on.

Proof. If the start v or end w is in neither G_A nor G_B or both in one of G_A or G_B , the problem can be resolved in $O(1)$ communication. So suppose $v \in G_A$ and $w \in G_B$.

Since G_A and G_B are complete, then a valid path occurs if and only if $V(G_A) \cap V(G_B) \neq \emptyset$. Then, by a similar argument to Claim 4, this problem is at least as hard as DISJ. Thus $D(\text{CTRAV}_2) \geq D(\text{DISJ})$ and so on. □

This tells us that the local structure of G_A and G_B alone is not a significant source of communication difficulty. That is, even when G_A and G_B are complete graphs, we do not get obviously better lower bounds.

Definition 2. Define STRAV_2 as a TRAV_2 problem where $E(G) = O(n)$, $E(G_A) = O(n)$, and $E(G_B) = O(n)$. In other words, the graph G is sparse and G_A, G_B are sparse too.

Claim 6. $D(\text{STRAV}_2) \geq D(\text{DISJ})$ and so on.

Proof. Consider the case where G_A and G_B are connected. If the start v or end w is in neither G_A nor G_B or both in one of G_A or G_B , the problem can be resolved in $O(1)$ communication. So suppose $v \in G_A$ and $w \in G_B$.

Then a valid path occurs if and only if $V(G_A) \cap V(G_B) \neq \emptyset$. Then, by a similar argument to Claim 4, this problem is at least as hard as DISJ. Thus $D(\text{STRAV}_2) \geq D(\text{DISJ})$ and so on. $\square$

This tells us that weak edge sparsity of G , G_A , and G_B does not add nor remove significant communication cost.

3 The Iterated Two-Party Traversal Problem

In section 2, we considered TRAV_2 , the communication game where both Alice and Bob computed the minimum length path from v to w under some conditions. However, many problems that are modelled by TRAV_2 tend to occur *iteratively*.

For example, consider the motivating problem 1.1. A user may want to send multiple queries in succession, while having partial information of the state of the system based on their previous queries. We try to generalize this notion here.

Definition 3. Let l-TRAV_2 be the following communication game: A graph $G = (V, E)$ is known, and two players Alice and Bob receive initial subgraphs G_A^0 and G_B^0 of G . Call this iteration $t = 0$. Alice and Bob then play TRAV_2 on G , G_A^0 , and G_B^0 . Then, some process p will transform G_A^0 into G_A^1 , and $G_B^1 = p(G_B^0)$. Then Alice and Bob play TRAV_2 on G , G_A^1 , and G_B^1 . This process continues for an indefinite number of iterations t .

Thus, a game of l-TRAV is an infinite iteration of TRAV_2 games on some graph G with subgraphs changing between iterations by some process p . There are two measures for the communication complexity of $\text{l-TRAV}_2(G, G_A^0, G_B^0, p)$: First, the initial communication cost, i.e. the communication at iteration 0, and the iterative communication cost, i.e. the communication between one iteration and the next.

Example 1. Suppose p is a process that constructs G_A^{t+1} from G_A^t as follows: add one vertex $v \notin G_A^t$ and all edges $(v, u) \in E(G)$ with $(v, u) \notin E(G_A^t)$, $\forall u \in V(G_A^t)$ to create H_A^t . Then delete one vertex $v' \in H_A^t$ and all edges $(v', u') \in E(G)$ with $(v', u') \notin E(H_A^t)$, $\forall u' \in V(H_A^t)$. This finally creates G_A^{t+1} . The choice of vertex is uniformly random in both cases. The same applies for G_B^t .

Then if in the initial iteration Alice and Bob send all of their inputs, every iteration afterwards Alice and Bob can solve TRAV_2 with only $4 \log(n)$ additional communication, where Alice communicates the vertices added and removed and Bob does the same.

The rest of this section is under construction!

4 The Multi-Party Traversal Problem

In section 2, we considered TRAV_2 , the communication game where both Alice and Bob computed the minimum length path from v to w using only edges in their subgraphs G_A and G_B . Here, we generalize the problem to TRAV_k , with k players.

The rest of this section is under construction!

5 The Iterated Multi-Party Traversal Problem

In section 3, we studied I-TRAV_2 , the iterative version of TRAV_2 . Here, we study the iterated version of TRAV_k , namely I-TRAV_k .

The rest of this section is under construction!

6 Other Problems

In the previous sections, we studied the TRAV and I-TRAV class of problems in great detail. However, there are other problems pertaining to multi-party graph traversal that may be of interest to study, either as defunct examples (where solutions can be found without communicating) or where the resulting bounds are not interesting enough to merit further study. Parts of this section is under construction!

6.1 Unbounded Traversal

Claim 7. *Suppose Alice and Bob access to the same information about graph G . Then the communication cost for any problem based on this alone is 0.*

One natural situation for this is in traversal, where Alice and Bob act as agents on a graph, and can traverse the graph to discover vertices and edges. If Alice and Bob start in the same connected component, as each has infinite compute power, each can explore the entire component and solve the problem independently.

References

- Rao, Anup, and Amir Yehudayoff. 2020. *Communication Complexity: and Applications*. Cambridge University Press
- Roughgarden, Tim. 2015. “Communication Complexity (for Algorithm Designers).” [\[Link\]](#)

Appendices

A Multi-Party Graph Traversal Case Studies

Here, we survey a variety of real-world case studies where communication was a major bottleneck. Note that this section is a work in progress, especially in terms of formal citations.

Large-scale transportation and infrastructure disruptions create a natural model for multi-party graph traversal. Consider the 2010 European volcanic ash shutdown: air-navigation service providers, airports, and airlines each control disjoint portions of airspace and schedules. Rerouting passengers is a shortest-path problem in a layered graph whose feasible vertices and edges change over time. No party can broadcast its full real-time state, both for operational and privacy reasons, so route feasibility must be computed with limited information exchange.

Similar graph-traversal constraints appear in urban transit shutdowns and phased restorations, where distinct agencies own stations and tunnels. After Hurricane Sandy, service availability evolved over weeks, and agencies coordinated alternate routes under partial knowledge of what infrastructure was open. Highway detours after the I-95 bridge collapse in Philadelphia illustrate the same phenomenon in a road network: different operators own highway segments and toll roads, and a single failed cut edge forces traffic onto alternative paths with asymmetric knowledge of capacities.

The maritime and logistics examples are equally instructive. The 2021 Suez Canal blockage forced carriers to reroute around the Cape of Good Hope or wait, a decision that depends on schedule and capacity information held privately by shipping lines and port operators. When the Baltimore Key Bridge collapsed, access to the port was blocked and cargo was diverted to alternate ports, again requiring coordination across a sea–port–land graph with private operational constraints.

Communication bottlenecks become even sharper in critical infrastructure networks. During the 2021 Texas ERCOT blackout, grid operators and utilities effectively solve a constrained flow and connectivity problem as generation and transmission nodes go offline. The Colonial Pipeline cyber disruption similarly forced fuel logistics to reroute through alternate depots and terminals with incomplete visibility. Finally, the 2021 Meta backbone withdrawal and the 2016 Dyn DNS attack show that digital services rely on reachability in routing and dependency graphs, where policy and security constraints limit information sharing across administrative domains.

These cases motivate a theoretical study of multi-party graph traversal: agents must collaboratively decide reachability and shortest paths while their private subgraphs, capacities,

or weights are not fully shareable. The fundamental question is how much communication is necessary, and how randomized protocols, sketches, or partial disclosures can reduce the burden while preserving correctness.

B Weekly Contributions

We summarize each member’s weekly contributions below:

Table A 1: Weekly contributions by team member.

Week	Ryan Batubara	Ivy Hawks	Darren Ho	Ciro Zhang
6	Work on formally writing up proofs for the trivial and other problems for our final report.	Work on formally writing up the proofs for the two party graph traversal problems.	Work on formally writing up the problem statement and motivation for the problem for our final report.	Work on formally writing up the introduction/motivation for communication complexity for our final report.
5	Help Ivy make the website. Write Github actions to publish to Github Pages. Make subposts components. Wrote CSS.	Start a Github pages website with our latest findings. Convert latex to markdown using Pandoc.	Write up formalized equality reduction in the report. Include the updated simplified procedure from the Professor.	Integrate all notes so far into the report. Fix typos etc in the other people who are writing the report. Give formal definitions, research sections.
4	Introduction section for report, defining communication complexity and various applications of problems.	Look into embedding 3SAT into graph problems. Useful to see techniques to embed hard problems into our problem, especially if we want to prove it is in NP.	Look deeper into random protocols. In particular, develop a bank of random protocols that we can embed into our problem.	Draw figures of our problem and embeddings in LaTeX for the report and poster.
3	Looked at extreme case examples in real-life transport networks, e.g. failure of a node, failure of a “central hub” e.g. in the case on natural disasters, etc.	Wrote down problem statement and preliminary problem solution in latex using hypergraph.	Presented what random protocols are and several different random protocols, namely private and public randomness.	Wrote notes on various variations of shortest path hypergraph problems.

Week	Ryan Batubara	Ivy Hawks	Darren Ho	Ciro Zhang
2	Identified various problems that involved the readings that we have found so far. Determined multiparty communication networks with self-driving cars shows promise. Showed how set disjointness can be used to determine collisions with self-driving cars. Also identified a protocol to compute this. Formalized potential problem.	Presented two papers on communication complexity of graph properties to see if there is any information that could be applied to self driving cars. Also presented problems that we will begin working on, with identifying a class of problems where the communication complexity is $O(n)$ by using the blackboard or third party model. Formalized potential problem.	Presented two papers on communication complexity of graph problems. One of them was on determining graph cliques, which can be used to determine whether or not there exists a clique, or a complete network. The other was to show a potential idea on how we can use ideas from other fields of study to compare with communication complexity. Took notes.	Presented a referee model, which we deduced to be the same as the blackboard model. We determined that this idea does not really work for our use case. Identified a critical flaw within our idea that was addressed. Found that our problem was too trivial and by removing/increasing certain assumptions the results remained trivial.
1	Drafted communication problems with multiparty graph traversal avoiding intersections. Attempted hidden matching problem communication complexity but determined that it would be trivial by having each person explore all nodes.	Drafted communication problems with multiparty weighted graph intersection. Determined that the majority of problems and solutions involved trivial solutions.	Drafted communication problems with regards to black hole problems where there is a node that eliminates all players. Determined that finding it would be trivial by only communicating after 2 or n moves. Recorded meeting notes.	Drafted communication problems where everyone knows about their distance from each other but not any information about the graph. Determined that it would be trivial.

C Updated Proposal

We take a graph, upon which there are players who wish to identify if the graph possesses some property. We wish to use this model to find the minimum communication required for each player to identify if this property exists. Each Player will receive some portion of the graph as their input—this can be either vertices, edges, or both. We mostly study reachability of vertices, in which the only paths considered valid are paths which use only the portions of the graph given to the players as inputs.

For motivation, please see Section 1 of the report.

C.1 Project Goals

Sections 2, 3, 4, and 5 formally define our problem statement and the full spectrum of problems we are considering for our project.

Clearly, section 2 is done. Much of the results for the remaining sections are also complete, and it only remains to formalize and type them up in $\text{\LaTeX}$.

C.2 Actionables

Therefore, this project can be summarized into three specific, actionables:

1. Finish writing this paper to outline the formal, technical communication complexity bounds for a given multi-agent traversal problem with certain constraints.
2. Finish porting the report to our website, whose link is available on the cover page of this report.
3. Give a poster presentation of the key results of the paper, with the intention of making the results applicable to a wider audience.

Therefore, our work will remain theoretically grounded while remaining applicable for the wider computing community.

C.3 Deliverables

We have three main deliverables:

1. A paper presenting actionable item 1 and 2.
2. A research poster of our key results, for actionable 3.
3. A website containing the paper and poster information, for actionable 3.

To be more specific, the paper will take the form of a research paper, where we define the

problems we are working on, prove bounds on these problems, and show how they model situations in the field of multi-agent graph traversal. The paper is our primary project output.

Our detailed weekly progress is shown in the previous appendix section. In the remaining time, we plan for the following actionables for each person:

Table A 2: Weekly plan

Week	Ryan Batubara	Ivy Hawks	Darren Ho	Ciro Zhang
7	Finish the other problems section, with a focus on figures.	Finish the iterated multi-party section, with a focus on figures.	Make significant progress on the multi party section, with a focus on figures.	Make significant progress on the iterated multi-party section, with a focus on figures.
8	Finish the multi-party section.	Finish the iterated multi-party section.	Finish the website port.	Move figures to first draft of poster.
9	Polish paper and proofread website.	Polish website and proofread poster.	Polish poster and proofread paper.	Polish poster and proofread paper.

Indeed, given our weekly contributions thus far we are on track to finishing the project with a week dedicated solely to polishing.